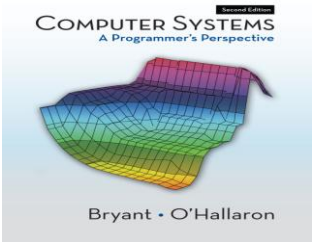
PRÁCTICA

➢ Pasado Continuo

Traduzca al Español las siguientes oraciones:

- “What were you doing at this time yesterday?” “I was studying for an exam.”
- “Was Carol at work last night?” “Yes, she was finishing her final report”.
- When I was young, I wanted to be an Information Systems Engineer.
- Jane was waiting for me at the company when I arrived.
- My computer was running slow, but I found out how to speed it up and make it safer.
- So there I was trying to find a job through the many online job posting sites but every time I tried to email my resume to a potential employer my computer would freeze. Or when I was searching for jobs this slow computer would make a simple search take an eternity.

➢ Futuro: will / going to



Computer Systems: A Programmer's Perspective, 2/E (CS:APP2e) by Randal E. Bryant and David R. O'Hallaron, Carnegie Mellon University

http://csapp.cs.cmu.edu/public/ch1-preview.pdf

Lea el siguiente texto correspondiente a la introducción del Capítulo 1 del libro arriba mencionado y enumere las habilidades que - según el autor - se podrán adquirir a partir de la lectura del libro (párrafo 3).

Chapter 1

A Tour of Computer Systems

A *computer system* consists of hardware and systems software that work together to run application programs. Specific implementations of systems change over time, but the underlying concepts do not. All computer systems have similar hardware and software components that perform similar functions. This book is written for programmers who want to get better at their craft by understanding how these components work and how they affect the correctness and performance of their programs.

You are poised for an exciting journey. If you dedicate yourself to learning the concepts in this book, then you will be on your way to becoming a rare "power programmer," enlightened by an understanding of the underlying computer system and its impact on your application programs.

You are going to learn practical skills such as how to avoid strange numerical errors caused by the way that computers represent numbers. You will learn how to optimize your C code by using clever tricks that exploit the designs of modern processors and memory systems. You will learn how the compiler implements procedure calls and how to use this knowledge to avoid the security holes from buffer overflow vulnerabilities that plague network and Internet software. You will learn how to recognize and avoid the nasty errors during linking that confound the average programmer. You will learn how to write your own Unix shell, your own dynamic storage allocation package, and even your own Web server. You will learn the promises and pitfalls of concurrency, a topic of increasing importance as multiple processor cores are integrated onto single chips.

In their classic text on the C programming language [58], Kernighan and Ritchie introduce readers to C using the `hello` program shown in Figure 1.1. Although `hello` is a very simple program, every major part of the system must work in concert in order for it to run to completion. In a sense, the goal of this book is to help you understand what happens and why, when you run `hello` on your system.

We begin our study of systems by tracing the lifetime of the `hello` program, from the time it is created by a programmer, until it runs on a system, prints its simple message, and terminates. As we follow the lifetime of the program, we will briefly introduce the key concepts, terminology, and components that come into play. Later chapters will expand on these ideas.